

Recursion Study Notes

Recursion is a way of thinking about problems: solve a large instance by solving smaller instances of the same problem. When you write a recursive function, do not try to trace every call in your head. Instead, answer two questions first: what is the decomposition rule, and what is the base case? Once those are settled, the call stack does the rest.

Beginners often try to mentally expand the whole recursion, lose track after three levels, and conclude that recursion is hard. A better habit is the recursive leap of faith: assume the function already works for smaller inputs, and verify only the step that combines results. This is induction, expressed as code.

1. Decomposition and base cases

The first line of a recursive function is almost always the base case. It answers the question: at what scale can the answer be returned directly? For factorial, the answer is immediate when n is 0 or 1. For directory traversal, a plain file is handled on the spot.

The recursive case answers a different question: how do we carve a problem of size n into strictly smaller subproblems? The crucial constraint is progress. Every call must move toward the base case, or the stack grows forever and the program dies with a stack overflow.

After writing both parts, check termination explicitly. Is there an input path that never reaches a base case? Factorial of a negative number is the classic counterexample, so robust implementations reject invalid input before recursing.

2. The call stack

Every recursive call pushes a new stack frame holding parameters, local variables, and the return address. Recursion depth is the number of frames alive at once. Once you internalize this, stack overflow stops being mysterious: the stack is a bounded resource, and depth past the limit aborts the program.

Rewriting recursion as iteration means maintaining an explicit stack of your own. You trade an invisible mechanism provided by the runtime for a visible data structure on the heap. The space cost is often similar, but the depth limit moves from the stack to memory, which is usually far more generous.

3. Recursion and induction

Induction and recursion are mirror images. A proof by induction checks a base case and an inductive step; a recursive function checks a base case and a decomposition rule. The certainty you feel about induction is the certainty you are allowed to feel about recursion, no more and no less.

A useful exercise: write factorial three times, then directory traversal twice, then the Tower of Hanoi once. Difficulty rises, but the checklist never changes. When you can explain your Hanoi solution out loud without notes, the concept has clicked.

4. What tail calls teach us

A recursive call is a tail call when it is the final action of the function and its result is returned unchanged. Tail calls can in principle be compiled into a jump, reusing the current frame. Mainstream JavaScript engines never shipped this optimization, so treat tail position as a design smell that a loop is easy to write, not as a performance promise.

Recursion is not a trick. It is the most natural notation for trees, divide and conquer, and backtracking search. The next note applies it to a real tree traversal, and draws every call on paper.